11376/8890 Computer Vision and Image Analysis - Computer Laboratory

Week 6 - Image Matching and Segmentation Techniques

[20/Mar/2026 Dr Min Wang]

Overview

In this lab, you will explore three fundamental computer vision techniques:

- Template Matching using Similarity Measures (SAD, SSD, CC, NCC)
- Feature-Based Matching (SIFT)
- Image Segmentation using Hough Transform (Line Detection)

These techniques form the foundation of many applications such as object detection, segmentation, tracking, and scene understanding.

Part 1: Image Template Matching

Objective

- Understand sliding window matching
- Implement different similarity measures:
  - SAD
  - SSD
  - Cross-Correlation (CC)
  - Normalized Cross-Correlation (NCC)

Concept

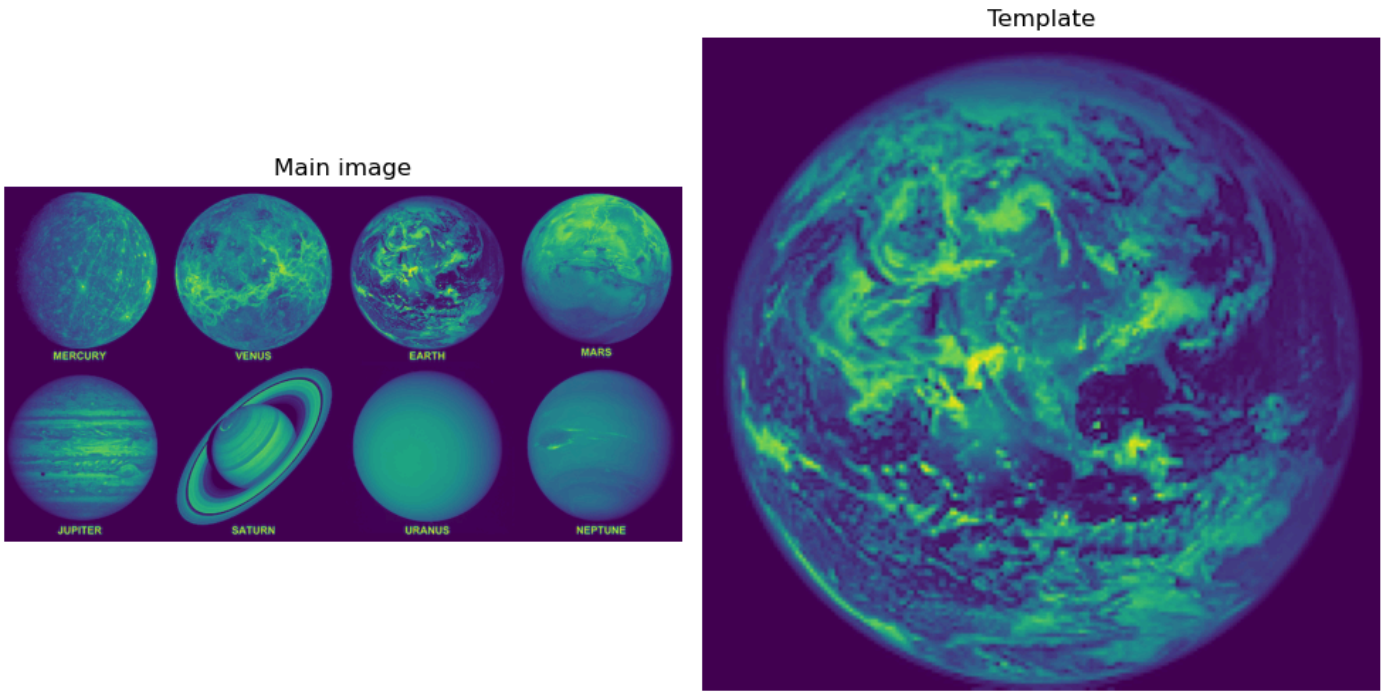
- Template matching compares a small template image with different regions of a larger image.
- At each location:
  - Compute a similarity score
  - Select the best match

Implementation

- Step 1: We first load images, our main image and the template image. Here both images are loaded in grayscale. Template size is needed to draw bounding boxes.

In [57]:
```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# Load main image and template
img = cv2.imread('solar system.png', 0)
template = cv2.imread('earth.png', 0)

# Plot the images
plt.figure(figsize=(10,8))
plt.subplot(1,2,1)
plt.imshow(img)
plt.title('Main image')
plt.axis('off')

plt.subplot(1,2,2)
plt.imshow(template)
plt.title('Template')
plt.axis('off')
```

```
plt.tight_layout()
plt.show()
```



- Step 2: Then we define the matching mathods, which are SAD (Sum of Absolute Difference), SSD (Sum of Squared Difference), CC (Cross-correlation), and NCC (Normalised Cross-correlation).

  OpenCV provides built-in approximations:

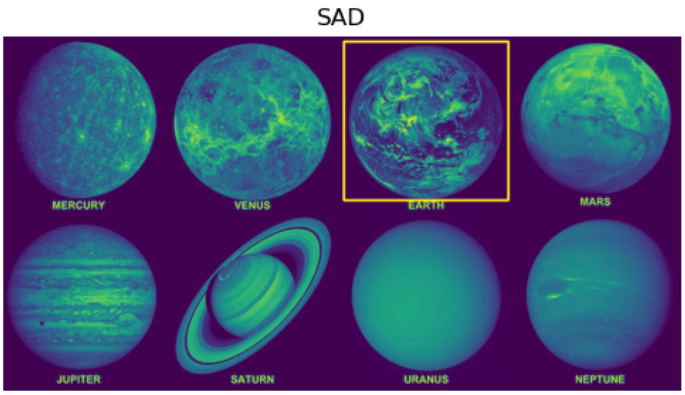
  - SQDIFF → difference-based (SAD/SSD) → minimize difference
  - CCORR → correlation → maximize similarity
  - CCOEFF → normalized correlation → maximize similarity, NCC is more robust to illumination changes

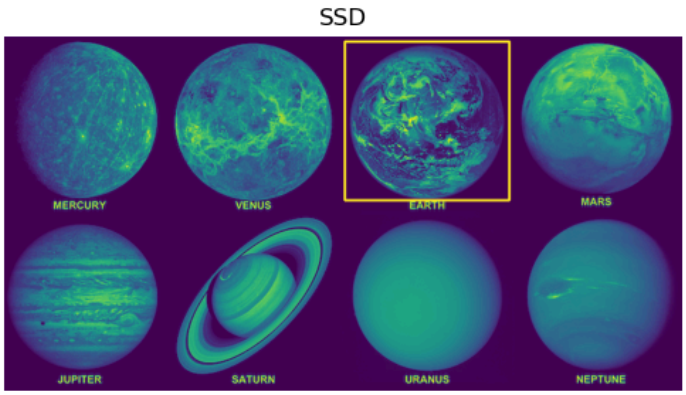
In [58]:
```
# Get template size
h, w = template.shape

# Here we define the Marching Methods
methods = {
    "SAD": cv2.TM_SQDIFF,
    "SSD": cv2.TM_SQDIFF_NORMED,
    "CC": cv2.TM_CCORR,
    "NCC": cv2.TM_CCOEFF_NORMED
}
```

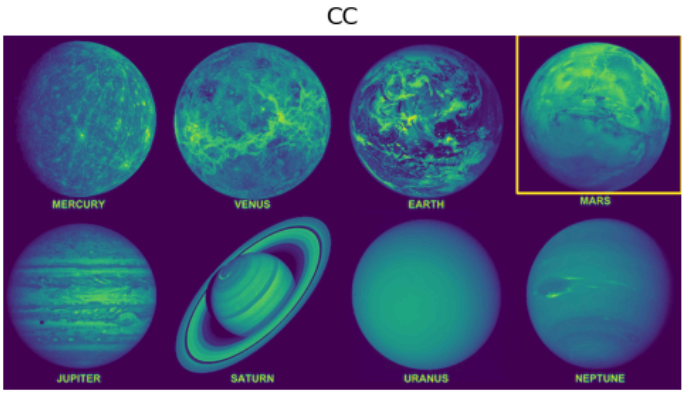
- Step 3: Perform template matching
  - Similarity Map: cv2.matchTemplate() slides the template across the image. At each position, it computes a similarity score. Output = a heatmap (result matrix).
  - Locate Best Match: cv2.minMaxLoc() finds minimum value (best for SAD/SSD), and maximum value (best for CC/NCC).
  - Bounding Box: The detected location is the top-left corner, We add template width & height to get the rectangle.
  - Visualisation: Draw rectangle to show where the template is found.

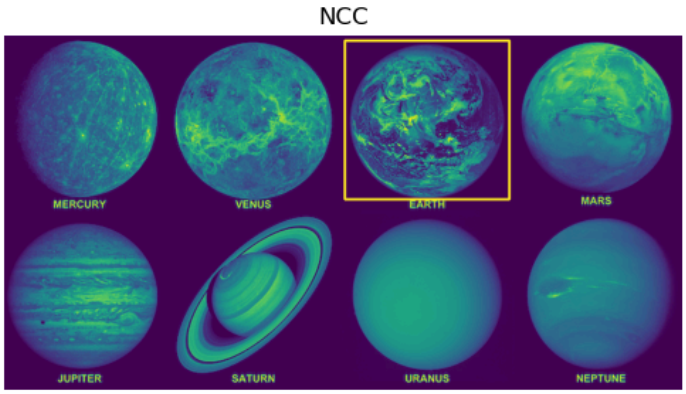
In [59]:
```
# Perform Template Matching
plt.figure(figsize=(10,8))

for i, (name, method) in enumerate(methods.items()):

    # Slide template over image
    result = cv2.matchTemplate(img, template, method)

    # Find best match
    min_val, max_val, min_loc, max_loc = cv2.minMaxLoc(result)

    # Choose location depending on method
    if method in [cv2.TM_SQDIFF, cv2.TM_SQDIFF_NORMED]:
```

```
        top_left = min_loc   # lower is better
    else:
        top_left = max_loc   # higher is better

    bottom_right = (top_left[0] + w, top_left[1] + h)

    # Draw rectangle
    img_copy = img.copy()
    cv2.rectangle(img_copy, top_left, bottom_right, 255, 2)

    # Display
    plt.subplot(2,2,i+1)
    plt.imshow(img_copy)
    plt.title(name)
    plt.axis('off')

plt.tight_layout()
plt.show()
```









Discussion

- Why does SAD/SSD prefer minimum values?
- Why is NCC more robust to lighting changes?
- What happens if the object changes scale?

Part 2: Feature-Based Matching (SIFT)

Objective

We implement local feature-based methods for matching images. Here we use Scale-Invariant Feature Transform (SIFT) features.

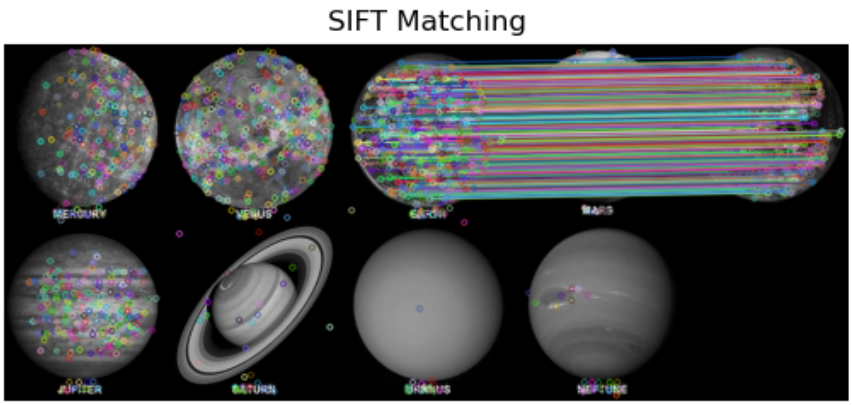
Key Idea

Instead of sliding windows, we:

- Detect keypoints

- Extract descriptors
- Match descriptors

In [60]:
```python
import cv2
import matplotlib.pyplot as plt

# Load the two images
img1 = cv2.imread('solar system.png', 0)
img2 = cv2.imread('earth.png', 0)

# Detect keypoints and compute descriptors
sift = cv2.SIFT_create()

kp1, des1 = sift.detectAndCompute(img1, None)
kp2, des2 = sift.detectAndCompute(img2, None)

# Match descriptors using KNN
bf = cv2.BFMatcher()
matches = bf.knnMatch(des1, des2, k=2)

# Ratio test
good = []
for m, n in matches:
    if m.distance < 0.5 * n.distance:
        good.append(m)

result = cv2.drawMatches(img1, kp1, img2, kp2, good, None)

plt.imshow(result)
plt.title("SIFT Matching")
plt.axis('off')
plt.show()
```



Explanation

kp (keypoints) contains the locations of the distinctive features. Each keypoint includes:

- position (x, y)
- scale
- orientation

des (descriptors):

- A 128-dimensional vector
- Encodes local gradient information around the keypoint

Part 3: Image Segmentation using Hough Transform

Objective

In this example, we detect straight lines using the Hough Transform. We need to understand the parameter space voting.

Concept

Each edge point votes for possible lines in the paramter space (ρ, θ). Peaks in this space correspond to detected lines.

Key idea

A line in image space:

$y = mx + c$

is transformed into parameter space:

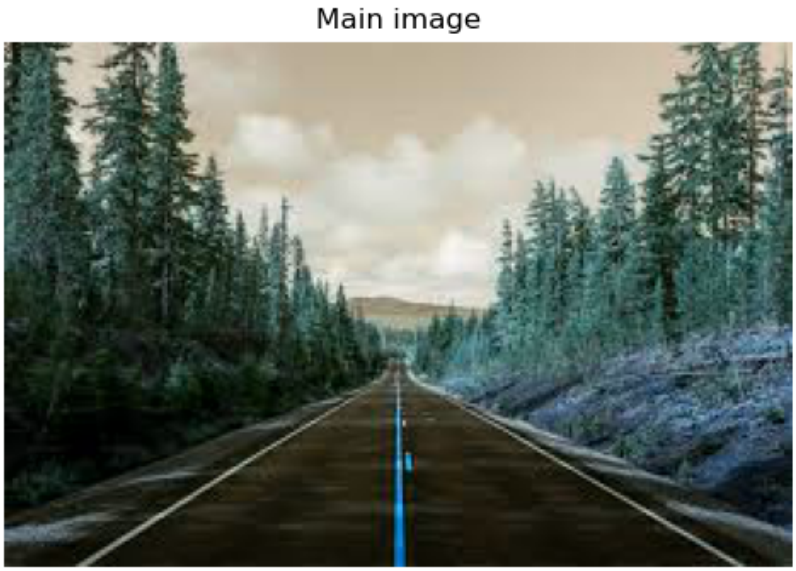
$\rho = x \cos\theta + y \sin\theta$

Each edge point votes in (ρ, θ) space.

Peaks → detected lines.

In [64]:
```
# Load images
img = cv2.imread('road.jpeg')
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

plt.figure(figsize=(10,4))
plt.imshow(img)
plt.title('Main image')
plt.axis('off')
```

Out[64]:
```
(np.float64(-0.5), np.float64(274.5), np.float64(182.5), np.float64(-0.5))
```

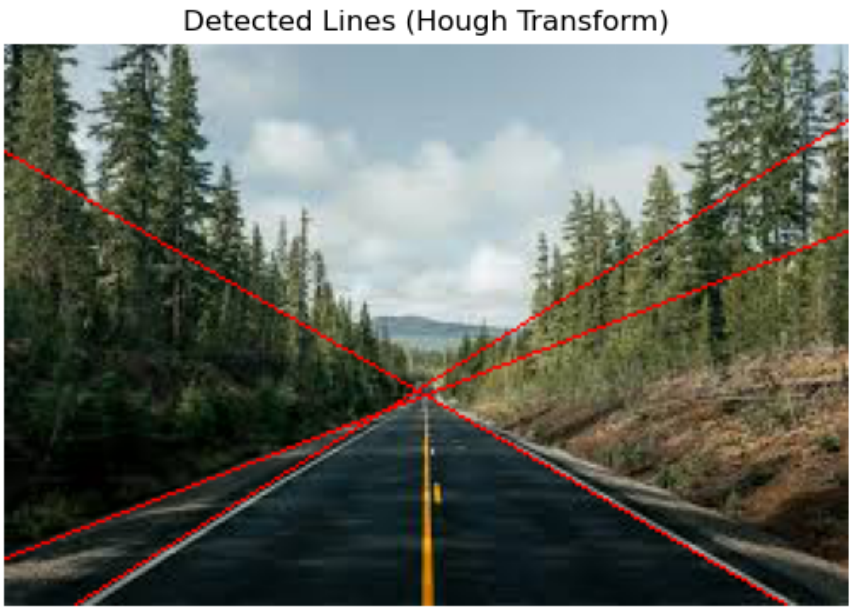


Next, we will:

- Step 1: apply the Canny Edge Detector to find edges
- Step 2: perform Hough Transform to vote in parameter space
- Step 3: detect peaks in parameter space -> identify lines

In [65]:
```
# Step 1: Edge detection
edges = cv2.Canny(gray, 50, 150)

# Step 2: Hough Transform
lines = cv2.HoughLines(edges, 1, np.pi/180, 135)

# Step 3: Draw lines
img_lines = img.copy()

if lines is not None:
    for rho, theta in lines[:,0]:
        a = np.cos(theta)
        b = np.sin(theta)
        x0 = a*rho
        y0 = b*rho

        x1 = int(x0 + 1000*(-b))
```

```
        y1 = int(y0 + 1000*(a))
        x2 = int(x0 - 1000*(-b))
        y2 = int(y0 - 1000*(a))

        cv2.line(img_lines, (x1,y1), (x2,y2), (0,0,255), 1)

# Display
plt.imshow(cv2.cvtColor(img_lines, cv2.COLOR_BGR2RGB))
plt.title("Detected Lines (Hough Transform)")
plt.axis('off')
plt.show()
```



Explanation

In the example code, we apply Canny Edge Detection to find edges, then apply Hough Transform to vote in parameter space, and finally detect Peaks to identify lines.

Important Parameters in cv2.HoughLines

rho: distance resolution theta: angular resolution threshold: minimum votes

Questions

- Why do we need edge detection before Hough?
- What does the threshold control?
- What happens if threshold is too low?

In [ ]: